



INDIA.RUNS

Build what next India runs on

Team Name : XYRON

Team Leader Name : Suhani Singh

Problem Statement : The Data & AI Challenge – Intelligent Candidate Discovery & Ranking

Solution Overview

- A hybrid candidate-ranking pipeline that scores every candidate in a 100K-profile pool against the JD using six weighted signals: TF-IDF semantic similarity, trust weighted skill matching, title/career-history fit, experience-band fit, location fit, and a behavioral-availability modifier – combined with a rule-based honeypot detector that suppresses subtly-fabricated profiles. The pipeline streams the dataset, scores each candidate independently, and keeps a bounded top-100 result set, producing a ranked CSV/XLSX with a grounded, per-candidate explanation.
- Traditional keyword search treats a buzzword-tagged skill the same as a deeply-used one. We weight every skill match by how long it was actually used and how many endorsements it received, and independently check career history against the JD's explicit disqualifiers (consulting-only careers, pure-research-only backgrounds, non-technical titles). A pure embedding-similarity model can still be fooled by keyword density; this hybrid design can't, because semantic similarity and rule-based checks fail independently of each other.

JD Understanding & Candidate Evaluation

- Production experience with embeddings-based retrieval (sentence-transformers, OpenAI embeddings, BGE/E5, and vector databases (Pinecone, Weaviate, Qdrant, FAISS, OpenSearch); strong Python; hands-on evaluation-framework design (NDCG, MRR, MAP, A/B testing); 6-8 years total experience with 4-5 years in applied ML/AI at product companies (not pure services); based in or willing to relocate to Pune/Noida. Nice-to-haves: LoRA/QLoRA/PEFT fine-tuning, learning to-rank, distributed systems.
- Skill trust weighting (proficiency × months actually used × endorsements, not just presence); career-history language for production-deployment evidence vs. pure-research-only signals; full employment history checked against disqualifiers (consulting-only career, CV/speech/robotics without NLP exposure); and behavioral signals — login recency, recruiter response rate, open-to-work flag, and notice period — used as an availability modifier, exactly as the JD instructs for down-weighting unavailable candidates.

Ranking Methodology

- The full candidate file is streamed twice: once to compute TF-IDF document-frequency statistics, once to score every candidate against the JD and maintain a fixed-size top-100 min-heap (via Python's `heapq`). No per-candidate API calls; no full dataset is ever held in memory or sorted in full.
- Hand-implemented TF-IDF + cosine similarity (pure Python standard library – no scikit-learn/scipy, to avoid compiled-binary dependency issues and maximize reproducibility); rule-based heuristics for skill trust, title/career fit, experience/location fit, and behavioral modifier; an ensemble honeypot detector (2-or-more independent red flags required to fire, the same principle used in fraud detection).
- Weighted linear combination: semantic similarity (0.20) + skill match (0.25) + title/career fit (0.25) + experience fit (0.10) + location fit (0.05) + behavioral modifier (0.15). Honeypot-flagged profiles have their composite score multiplied by 0.05 (suppressed, not deleted, for auditability) before final sorting.

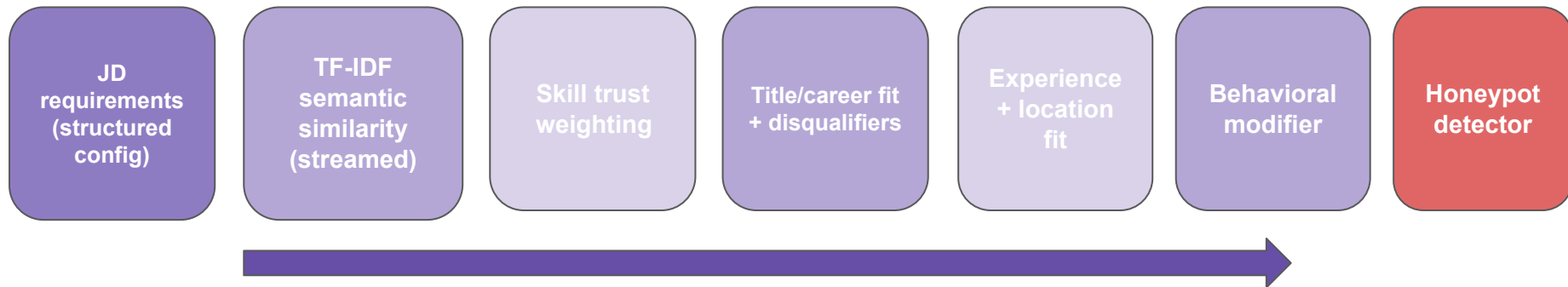
Explainability & Data Validation

- Each candidate's reasoning string is assembled directly from the same scored feature values used to rank them – matched core skills, title/career notes, behavioral notes, and location – never from a separate free-text generation step.
- Because reasoning text is templated from already-computed, already-verified fields (not generated by an LLM), nothing can be asserted in a candidate's explanation that wasn't actually present and matched in their own profile data.
- A rule-based honeypot detector flags experience-years vs. career-history-duration mismatches, 'expert'-level skills with near-zero months of actual use, multiple simultaneous full-time jobs, invalid education date ordering, and overlapping employment history. Two or more independent flags trigger a 95% score suppression.

End-to-End Workflow

- 1) JD is encoded once into a structured requirements config (must-have/nice-to-have skills, disqualifiers, location/experience preferences, component weights).
- 2) Pass 1 streams all candidates to build TF-IDF document-frequency statistics.
- 3) The JD is vectorized once against that vocabulary.
- 4) Pass 2 streams all candidates again: each candidate's semantic similarity, skill/title/experience/location/behavioral scores, and honeypot flags are computed independently. 5) Scores are combined into a weighted composite; a fixed-size top-100 heap is maintained throughout (no full sort of 100K rows).
- 6) The heap is sorted by final rounded score with a deterministic candidate-ID tiebreak, and written out as a ranked CSV/XLSX with grounded reasoning.
- 7) Output is checked against the official validator before submission .

System Architecture



Weighted composite score → bounded top-100 heap → ranked CSV/XLSX with reasoning

No GPU. No network calls. No third-party ML dependencies in the ranking step (pure Python standard library)

Results & Performance

- 0 honeypots in the top 100 (well under the 10% disqualification threshold). Output is fully deterministic across repeated runs on the same input. Top-ranked candidates show legitimate technical titles, core skills backed by real usage history, and evidence of production deployment – directly reflecting the JD's stated priorities. Output passes the official `validate_submission.py` with zero errors.
- Full 100K-candidate run completes in roughly 45-55 seconds on a standard CPU laptop – well under the 5-minute budget. Zero GPU usage, zero network calls during ranking, and zero third-party ML dependencies (pure Python standard library for the ranking engine), specifically to maximize reproducibility across different grading environments. Memory stays bounded via a two-pass streaming design with a fixed-size top-100 heap, rather than materializing the full scored dataset.

Technologies Used

- Python 3 standard library only (re, math, collections, heapq, csv, json) for the core ranking engine — chosen specifically to avoid scikit-learn/scipy's compiled binary dependencies, which can be blocked by OS-level security policies (e.g. Windows Application Control) and which add reproducibility risk in graders' environments.
- openpyxl for the final ranked XLSX output.
- Streamlit + pandas for an optional sandbox demo app, deployed via Streamlit Community Cloud, separate from the core ranking pipeline.
- Git/GitHub for version control and submission.

Submission Assets

- **GitHub Repository:** [Aki7-web/redrob-ai](#)
- **Sandbox / Demo Link:** [Streamlit](#)
- **Approach Deck (this PDF):** included with submission
- **Ranked Output File:** submission.xlsx (top 100 candidates, validated)



INDIA.RUNS

Build what next India runs on

THANK YOU